

# Battleship Solitaire Solution Synthesis Using Rosette’s Angelic Execution

Satyam Awasthi

University of California, Santa Barbara

satyam@ucsb.edu

## Abstract

A logical puzzle is built on the premise to find the solution by making use of several clues and restrictions. We describe the Battleship (aka Battleships Solitaire aka Shipfind), a logical puzzle, and formulate it as a Constraint-Satisfaction-Problem (CSP). The problem has been attempted in a variety of ways including using genetic algorithms, supervised and reinforcement learning, and integer and constraint programming problem as well. However, as a machine learning problem, it would require a significant amount of resources to model, time to train and, run (3 minutes for [1]), and still not guarantee a solution. Moreover, the puzzle is quite a challenge to model successfully using constraint programming (CP)[2]. There are instances that existing rule-based software for Solitaire Battleships, Fathom It!, cannot solve [3]. In this work, we present another CSP model for the puzzle and lift Rosette [4], a solver-aided programming language. We show that the challenges faced when modeling the puzzle are addressed as we reduce the CSP to a set of solver-aided queries (angelic executions [4]) under well-defined assertions. Moreover, we try to quantify the perceived difficulty of the puzzle in terms of grid size and the number of ships.

**Keywords:** constraint-satisfaction-problem, angelic execution, non-determinism, solver-aided programming, symbolic evaluation, perceived difficulty

## 1 Introduction

One solves a logical puzzle by making use of several clues and restrictions. Several techniques have been used to solve puzzles, like Optimal algorithms for Mastermind are considered by Chen and Lin (2004) [5]. Or iterative algorithms in the Hanoi or Reves puzzles (Gedeon 1996, Majumdar 1994, Sapir 2004, Sniedovich 2002) [6,7,8,9]. In this project, we will use symbolic evaluation, the process by which Rosette turns a query into an SMT constraint, and non-determinism (angelic execution).

Rosette [4] supports the solver-aided query using non-determinism, which they call angelic execution. This query is the dual of verification. Given a program with symbolic values, it instructs the solver to find a binding for them that will cause the program to execute normally—that is, without any assumption or assertion failures.

Angelic execution can be used to solve puzzles, run incomplete code, or “invert” a program, by searching for inputs that produce the desired output.

Formally, `(solve expr)` searches for a model of the formula  $(vc\text{-}assumes\ P) \wedge (vc\text{-}asserts\ P) \wedge (vc\text{-}assumes\ Q) \wedge (vc\text{-}asserts\ Q)$ , where  $P$  is the verification condition before the call to solve and  $Q$  is the verification condition generated by evaluating `expr`.

Battleship is a find-the-fleet puzzle based on a grid representing an ocean section with a fleet of ships hidden inside. The goal is to reveal the fleet by logically deducing the location of the ship segment. Such puzzles often appear in puzzle magazines [10]. Wikipedia (Battleship puzzle 2009) [11], reports the game was invented in Argentina by Jaime Poniachik. It was first published in 1982 in the Spanish magazine Humor and Juegos. The game gained more attention and appears regularly in puzzle magazines after its international debut at the first World Puzzle Championship in New York City (Van de Liefvoort 1992). In most puzzle magazines, the sizes of these grids differ, where the smallest puzzle is of size  $5 \times 5$  and the largest puzzle is of size  $15 \times 15$ .

A Battleships puzzle consists of a puzzle grid, a column and row tally, and a fleet containing a certain number of ships of varying lengths. [10] enlists the sizes for them: a 4 units long battleship, 3 units long cruiser, a destroyer is 2 units, and a submarine is only 1 unit in length. We will assume that only the aforementioned kinds of ship in developing our solution. The initial grid is partially filled with ship segments or water (denoting the absence of a ship segment). The goal of a puzzle is to show that the provided ships can be positioned on the puzzle grid, meeting the following four conditions:

- C1: all ships in the fleet are put in the grid;
- C2: the indications in the initial grid are respected;
- C3: no two ships occupy adjacent (orthogonally or diagonally) squares;
- C4: the number of ship segments in column (row)  $i$  is equal to the  $i^{\text{th}}$  value of the column (row) tally.

Figure 1a shows an example of an unsolved solitary battleship puzzle that appeared in the “Detailed Solution Key for 2000 World Puzzle Championship Qualifying Exam, Problem 6,” 2000, accompanying the solution in Figure 1b.

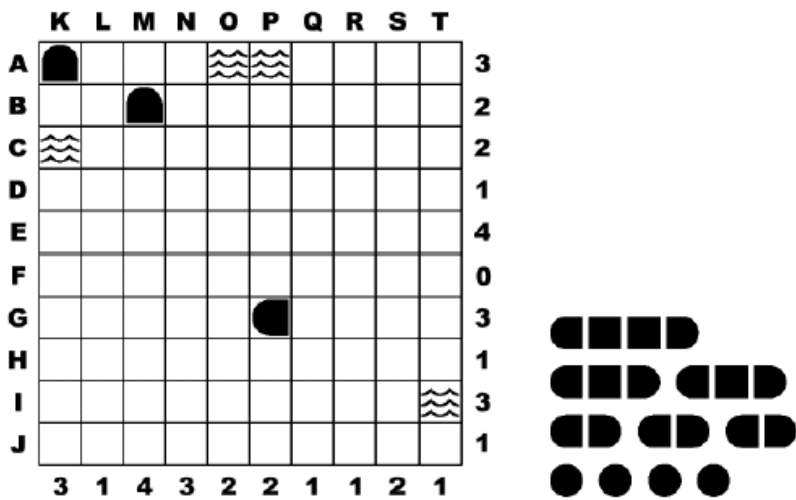


Fig. 1a. Example Solitaire Battleship Puzzle [12]

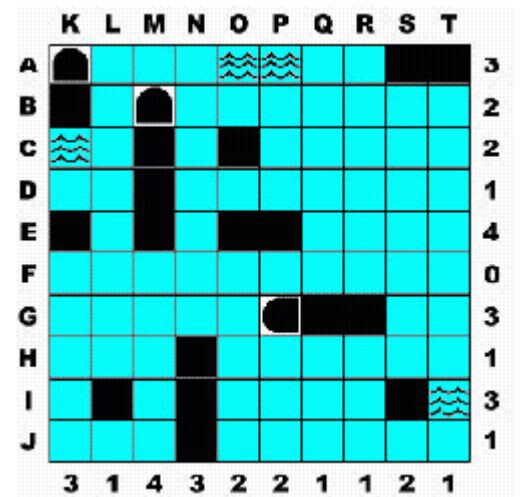


Fig. 1b. Solution to puzzle in Figure 1a [12]

## Code Available at GitHub

Our work, Battleship Solitaire Solution Synthesis using Rosette’s Angelic Execution, models the puzzle as a CSP and lifts Rosette [4] to solve it, specifically its feature of non-determinism. We have made our work available publicly and the source code can be found at <https://github.com/SatyamA007/CS292C-Battleship> [15]. The GitHub repository contains all the necessary information for understanding and reproducibility of our work.

## 2 Overview

We write down our key contributions in this section that capture the essence of our approach. Also, we will refer to our program module that solves the puzzle supplied to it as the “solution-synthesizer”. And, the module generates the puzzles, the “generator”. This terminology will be followed throughout the report. Figure 2 gives a comprehensive and intuitive outline of our work.

A user at the program interface gets to perform a solve or a generate action. Depending on his choice the respective module is activated.

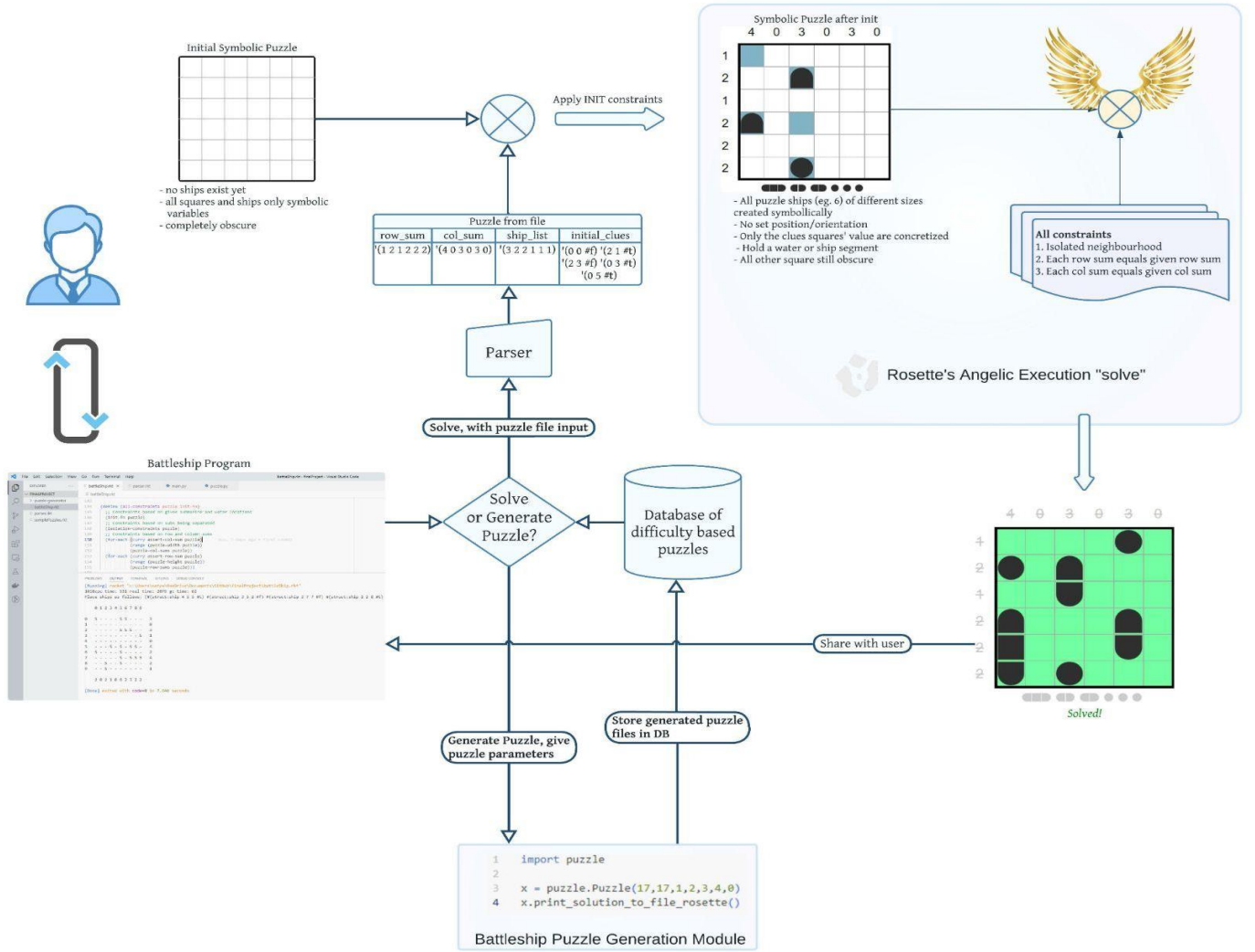


Figure 2: Overview of our Approach

In case a solver is triggered, a puzzle file must be supplied to it. By default, it is set to solve all the puzzles contained in the directory. The parser is the component that analyses a puzzle in the human-readable form and gets the precise constraints that fit the constraint scheme of our solution-synthesizer. The initialization phase follows - from an obscure state of uninitialized symbolic constants, the puzzle gets partially concretized for the grid squares that are provided as a clue. The information for all the ships is provided and includes their number and sizes. The position and orientation of each would fetch us the desired solution.

The next phase is Rosette's angelic execution and involves applying all the necessary constraints to the symbolic grid squares and ships. The "solve" query returns one model as part of a satisfiable solution if it exists; otherwise, the result is an unsatisfiable solution.

The following steps provide a brief summary of our strategy -

1. **Generate puzzles:** We develop an algorithm that, given the input parameters for the puzzle, will randomly place all the ships in the grid of size  $m \times n$  such that: it stays within the grid boundary, and doesn't overlap with any already ship placed. The total area of the grid is an effective metric for puzzle difficulty since it proportionally increases the number of variables considered
2. **Define the constraints:** M.Smith et al. [2] discuss a basic model for tackling the puzzle by developing a Constraint Programming (CP) model for it. We take note of the necessary constraints expressed by [2], which we adapt to build our model assertions

3. **Design the solver:** We maintain each square of the puzzle and ship as a symbolic variable. A binding of symbolic constants to concrete values exists if all the assumptions and assertions are satisfiable.
4. **Parse the constraints:** Interface the solver to different difficulty-based puzzles and precisely parse the constraints from them
5. **Synthesis:** Using Rosette’s Angelic Execution, search for a binding of symbolic constants to concrete values that satisfies all the assumptions and assertions encountered during and before the call to “solve” query

### 3 Problem Formulation

#### 3.1 Rules followed by our framework

To solve, all the necessary clues and restrictions are given by [10] along with the 4 conditions mentioned in section 2. We frame the rules for our solution-synthesizer that encapsulate them and, suits our framework.

**Rule 1:** the numbers on the right and on the bottom of the grid indicate how many squares in the corresponding row and column are occupied by ship segments.

The fleet traditionally consists of 10 ships. One battleship, four spaces long, two cruisers three spaces long, three destroyers two spaces long, and four submarines that take only one space.

**Rule 2:** Ships may be oriented horizontally or vertically without touching each other, not even diagonally.

And sometimes,

**Rule 3:** A few squares may be revealed to start you off. Revealed squares are fixed and the solution must incorporate these squares as is.

#### 3.2 Defining constraints

M. Smith et al. [2] discuss a basic model for tackling the puzzle by developing a Constraint Programming (CP) model for it. We take note of the necessary constraints expressed by [2], that we will use to build our model assertions.

Let the size of the grid be  $n \times n$  so that typically  $n = 10$ . For each square of the grid, variables  $s_{ij}$  are defined:

$s_{ij} = 1$  if the square  $(i, j)$  is occupied by a ship segment and 0 otherwise,  $\forall i, j, 0 \leq i, j \leq n + 1$ .

Thus, the primary constraints on these variables are:

- **Isolated Neighborhood**  
if  $s_{ij} = 1$ , then  $s_{i-1,j-1} = s_{i-1,j+1} = s_{i+1,j+1} = s_{i+1,j-1} = 0$ .
- **Row sum**  
 $\sum_j s_{i,j} = R_i$ , where  $R_i$  is the tally (number of occupied squares) for row  $i$ ;
- **Column Sum**  
 $\sum_i s_{ij} = C_j$ , where  $C_j$  is the tally for column  $j$ .

### 4 Interactive Synthesis Algorithm

In this section, we lay out the course of action for the approach we followed in the solution-synthesizer module that is consistent with sections 3.1 (Rules Followed) and 3.2 (Defined Constraints). In the interests of simplification, we break down our course of action into 3 phases.

#### 4.1 Puzzle Analysis and Parsing

A sample puzzle is shown in Figure 3. We designed the generator to store the puzzle solution within the puzzle file.

This allows us to later compare the two solutions available to us after completion of the solution-synthesizer: one already existing in the puzzle file, and the other materialized by the Rosette framework. This gives us an interesting

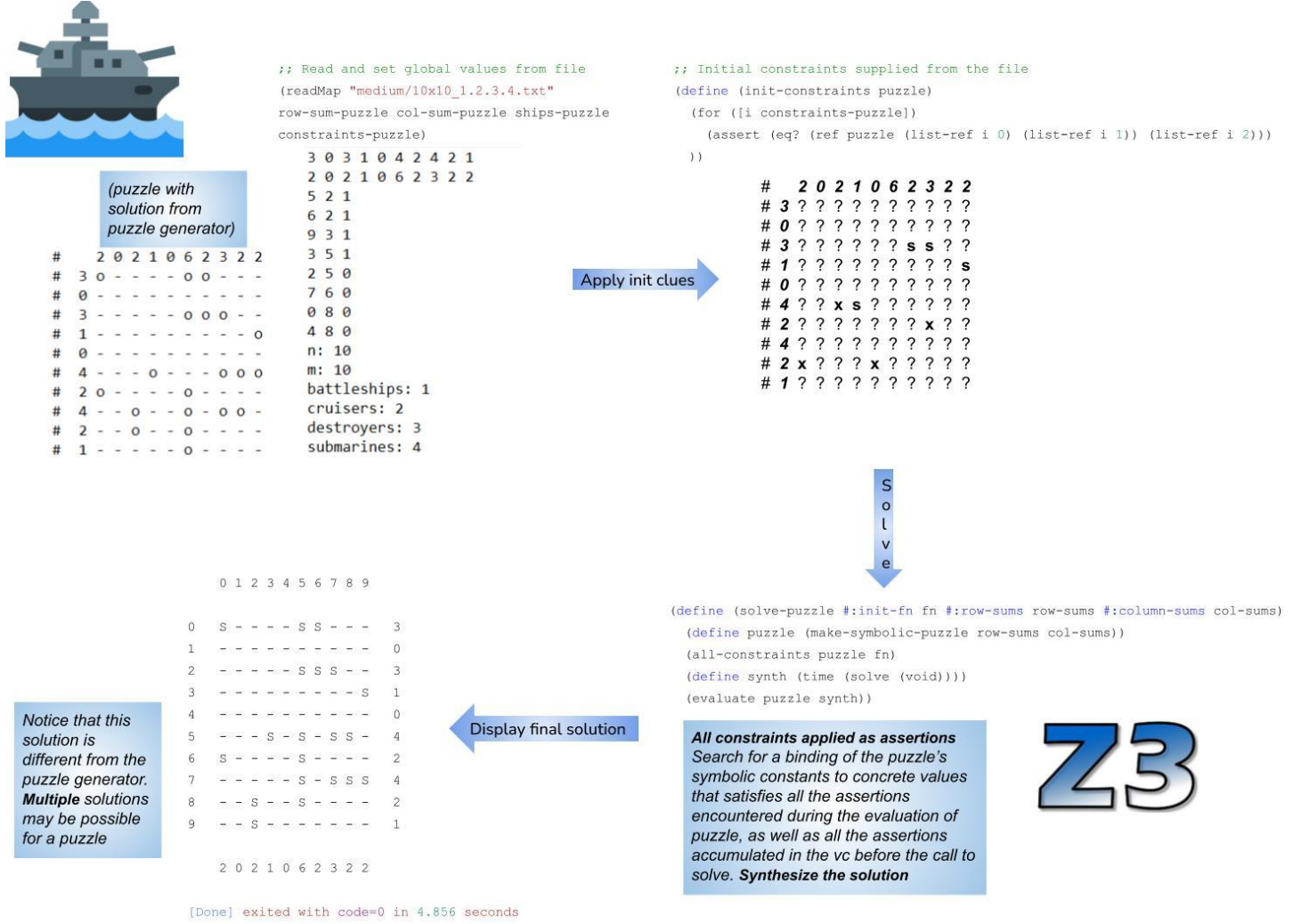


Figure 3: Course of Action for Interactive Synthesis Algorithm

insight that the two solutions are not always identical.

Our parser extracts 4 important components which are all we need to feed to our solution-synthesizer for a solution. (1) row-sum is the tally of ship segments for each row, (2) column-sum is the tally of ship segments for each column, (3) initial-clues in the format “x y 0/1” give the (x,y) location of the specified puzzle component which can be water (specify 0) or ship piece (specify 1), and (4) fleet of ships in the form “a1 a1 a2 ...”  $\forall a_i \in \mathbb{Z}$  is the length of a ship. For example, a fleet of “4 3 3 2 2 2 1” gives the typical setting of 1 battleship of length 4, 2 cruisers of length 3, 3 destroyers of length 2, and 3 submarines of unit length.

Puzzle or grid size is estimated from the row-sum and col-sum. Since the size n of the column array row-sum gives the number of columns, similarly we get the number of rows m from col-sum. Thus, the grid size is m x n.

## 4.2 Applying all constraints

Since we are using Rosette’s angelic execution, we apply all constraints by means of subjecting the symbolic variables in question to assertions. We model the constraint in section 3.2 to be applied as assertions while ensuring that rules in section 3.1 hold in each step. The first set of assertions applied is the “init-clues”. These effectively direct our solution-synthesizer to incorporate the specified uncovered parts (squares) of the puzzle into the final solution. Also, the puzzle size and the fleet will be initialized in our “symbolic” puzzle to contain information like the size and number of ships but no knowledge of their position or orientation is known. Thus, some squares will be concretized at this step and the

symbolic puzzle at this point would resemble one that can be attempted by a human. Next, we apply 3 sets of constraints as assertions in succession - (1) Isolated neighborhood, for  $s$  as the total number of ships, it is an  $O(s^2)$  step since each ship cannot be present in the 8-connected neighborhood (refer Figure 4) of every other ship.

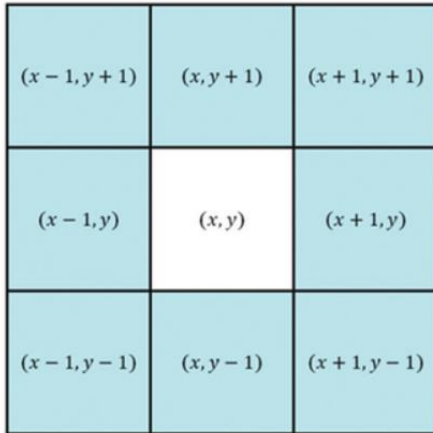


Figure 4: 8-pixel connected neighbors for  $(x,y)$  are blue [14]

(2) Row-sum, and (3) Column-Sum tallies are  $O(m \times n)$  complexity assertions since we assert the symbolic sum along each row or column must be the specified concrete value.

### 4.3 Rosette’s Angelic Execution, “Solve”

The angelic execution in Rosette [4] searches for a binding of the symbolic variables in our symbolic puzzle namely the squares  $s_{ij}$  and the ships in the fleet to concrete values that satisfy all the assumptions and assertions encountered during the evaluation of  $\text{expr}$ , as well as all the assumptions and assertions accumulated in the verification condition (vc) before the call to solve [4]. All the constraints we applied to the puzzle exist in the vc as accumulated assertions.

If such a binding exists, the query returns one (model) as part of a satisfiable solution. Otherwise, the result is an unsatisfiable solution. The model is displayed intelligibly on the terminal as shown in Figure 3 (display final solution). The ship pieces are marked “S” and the water locations are “-”, while the row-sum and col-sum are also shown along with the row and column indices next to the respective rows and columns.

## 5 Implementation

We will illuminate more on the details pertaining to encoding the solution-synthesizer and our experience of using the Rosette framework in this section. A chronological walkthrough of the code will be followed as it will also highlight the course of action. Moreover, it will serve as an informative guide when referring to our code.

```
(define difficulty-level "puzzle-generator/hard/")
```

The difficulty of puzzles is set from the above line. Specifically, it directs the solution-synthesizer to look into the given directory for puzzles and then proceeds to solve all of them one by one.

```
;; Read and set global values from file
(readMap puzzlePath row-sum-puzzle col-sum-puzzle ships-puzzle constraints-puzzle)
(set! ships-puzzle (unbox ships-puzzle))
(set! row-sum-puzzle (unbox row-sum-puzzle))
(set! col-sum-puzzle (unbox col-sum-puzzle))
(set! constraints-puzzle (unbox constraints-puzzle))
```

The `readMap` procedure calls the parser and extracts and puts the resulting constraints into box variables. Using `box`, `unbox`, and `set-box!` is very close to using an explicit pointer in C. This is necessary because Racket (a descendent of Scheme) does not support “pass by reference” construct for function calling that modifies our variables. Although this might not be in line with Scheme’s functional nature, we still opted for using it as a modular solution.



```
(solve-and-print-puzzle    #:init-fn init-constraints
                          #:row-sums row-sum-puzzle
                          #:column-sums col-sum-puzzle)
```

We begin with solving procedure “solve-and-print-puzzle” and supply the constraints as keyword arguments. The order of keyword args determines the order in which arg-exprs are evaluated, but a function accepts keyword arguments independent of their position in the argument list. It made using function calling more convenient since we often had to pass several arguments to a function ensuring exact positional order of the arguments was followed if not using keywords.

```
(define (solve-puzzle #:init-fn fn #:row-sums row-sums #:column-sums col-sums)
  (define puzzle (make-symbolic-puzzle row-sums col-sums))
  (all-constraints puzzle fn)
  (define synth (time (solve (void))))
  (evaluate puzzle synth))
```

The above block of code summarizes the objective of our solution-synthesizer in a few lines: First, it initializes the symbolic puzzle. Then, it applies all the 3 sets of constraints discussed in section 4 (applying constraints). Followed by the “solve” query for angelic execution. Next, the “evaluate” produces a new value obtained by replacing every symbolic constant *c* in the puzzle with (sol *c*) and simplifying the result.

```
(struct puzzle (height width row-sums col-sums ships matrix))
```

One of the appealing reasons to choose Rosette as a solver-aided language is its architecture allowed us to construct synthesizers without changing existing interpreters and type checkers, by just making their inputs symbolic [4]. The input can even be a user-defined abstract data type like the “puzzle” above, which Rosette was able to synthesize the solution for without any extra steps.

## 6 Evaluation

| BattleShip   | Destroyer | Cruiser | Submarine | Net Ship Area | Puzzle Size N | Time (in ms) |
|--------------|-----------|---------|-----------|---------------|---------------|--------------|
| 0            | 1         | 2       | 3         | 10            | 10            | 2339         |
| 1            | 2         | 3       | 4         | 20            | 10            | 6087         |
| 0            | 1         | 2       | 3         | 10            | 11            | 6506         |
| 1            | 2         | 3       | 4         | 20            | 11            | 38060        |
| 1            | 3         | 4       | 5         | 26            | 11            | 29152        |
| 0            | 1         | 2       | 3         | 10            | 12            | 26339        |
| 1            | 2         | 3       | 4         | 20            | 12            | 35878        |
| 1            | 3         | 4       | 5         | 26            | 12            | 92092        |
| 1            | 2         | 3       | 4         | 20            | 13            | 91268        |
| 1            | 3         | 4       | 5         | 26            | 13            | 116387       |
| Average Time |           |         |           |               |               | 44410.8      |

Table 1: Time taken to solve medium level puzzles of varying sizes and net ship area

Table 1 provides gives the total time taken by to solve medium level puzzles of varying sizes (square grids of size  $N = 10$  to 13) and different fleet sizes (net ship area). The problem set for puzzles only contained squares, though rectangular

puzzles can be similarly solved as well, as square grids make the arithmetic (used in this report) simpler to deal with. The specifications of the machine that ran the solution-synthesizer are Windows 10 PC (Ryzen 7 3750h, 16GB RAM) / ASUS ROG Zephyrus G 2019, a mediocre laptop.

Similar to the medium puzzles, 10 different puzzles were solved using the Rosette-powered solution-synthesizer. The average running times (in s) are 8.04(easy), 44.41(medium), and 517.59 (hard). The exponential increase in time gives us the intuition that the perceived difficulty of the puzzle would depend on the total number of symbolic constants involved (including the grid squares and the ships of the fleet). We thus set out to obtain a precursive measure of the perceived difficulty of the puzzle Battleship Solitaire.

## Perceived Difficulty

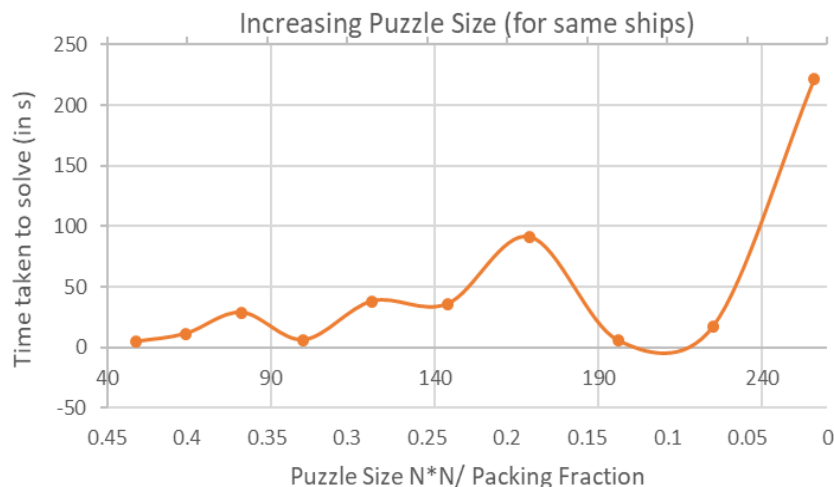


Figure 5: Time taken for increasing the puzzle sizes/decreasing the packing fraction and same fleet

We plot the time taken (in vertical axis) against the puzzle size ( $N \times N$ ) on the primary horizontal axis and packing fraction on the secondary horizontal axis. We define the packing fraction as the ratio of total space occupied by the fleet to the puzzle's total area. A smooth trend line is drawn, that depicts an overall increase in the time and so, the difficulty with increasing the puzzle size which is expected since we increase the total symbolic constants with  $N \times N$ . Also, we will use the term difficulty synonymously with the time taken to solve puzzle in the following section.

However, there are multiple local extrema in the curve: regions of high and lows. Hence, there are other factors devoting to the difficulty. Instinctively we can state that for a high packing fraction, we can only make so many moves. A ship placed in the partially solved puzzle is more likely to be readjusted and replaced to obtain the final solution. Unlike in a low packing fraction puzzle, where multiple solutions can exist, for a high packing fraction solution, the solution is often unique. Thus, we explore the effect of packing fraction on the difficulty.



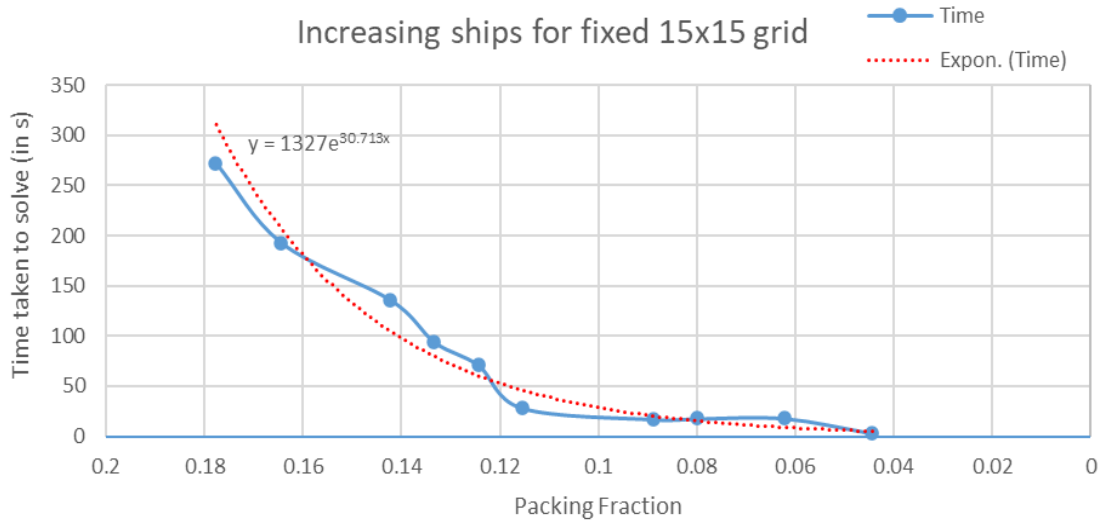


Figure 6: Effect of packing fraction on perceived difficulty

Figure 6 confirms our suspicion, the exponential trend line (in red dots) given by the equation

$$y = 1327e^{30.713x}$$

Which is also quite accurate in predicting the time taken by the solution-synthesizer, with an RMSE of 5.87 s and normalized RMSE of 0.069. So, we can say that this model captures the difficulty principles for this case: difficulty exponentially decreases with the packing fraction.

This explains the difficulty trend seen in Figure 5. As the puzzle size is increased, the packing fraction decrease and so, the associated difficulty component decreases. And, the increase of symbolic constants causes an overall increase in difficulty.

## 7 Related Work

The puzzle has been attempted in a variety of different ways. However, there are a few caveats to some of them: learning-based solutions are resource-intensive and do not guarantee a solution [1]. Rule-based solutions also don't always provide a deduction [3]. Although, CP-based guarantees a solution for valid instances, the modeling of the puzzle can be quite challenging [2].

Solitaire Battleship is approached as a decision problem by Sevenster [13] who concludes that the problem is NP-complete, but lists some explanations as to why humans often find the problems quite solvable.

Fathom It! (October 1997) [3], rule-based software for Solitaire Battleships. The underlying rule-based system will offer a deduction, given the current state of the board. But there are instances that existing rule-based software for Solitaire Battleships, Fathom It!, cannot solve [2].

Smith [2] attempts to solve puzzles using constraint satisfaction while avoiding any local search. Although not all problems were found to be solvable without search, this method seems very promising.

C. Port [1] uses a genetic algorithm and provides an improvement to that by developing a Wisdom of Artificial Crowds (WoAC) algorithm to combine multiple solutions to the battleship problem. It offers a higher success rate but takes a long-running time (3 minutes for [1]) and does not guarantee a solution.

## 8 Conclusion

The first conclusion is that utilization of the Rosette framework was a boon for designing a solver for the BattleShip Solitaire puzzle. Its architecture allowed us to construct the synthesizer without changing existing interpreters and type checkers, by just making their inputs symbolic. The input can even be a user-defined abstract data type like the “puzzle” above, which Rosette was able to synthesize the solution for without any extra steps. Moreover, our solution-synthesizer guarantees a solution, assuming the correctness of the given puzzle, and does so in less time compared to existing approaches (section 7). Rosette allowed us to map the result of constraint solving back to the program, which the program used to update the puzzle state, and generate other constraints. Encoding the constraints was also a straightforward exercise.

We observed that there was an exponential increase in the time taken or difficulty of the puzzle for increasing the size of the puzzle. For our solution-synthesizer, this is because it causes a similar increase in the number of symbolic constants, which might cause an exponential increase for the Z3 solver present under the hood in the Rosette framework. We realize that due to this fact, the time complexities for applying assertions do not directly map to real-time.

Finally, in section 6, we tried to estimate a metric for perceived difficulty. And found that, both the puzzle size and packing fraction contribute directly to it. In fact, we found a noteworthy trend estimate for difficulty depending on packing fraction. This somewhat explained the local extrema that occurred in Figure 5. However, more analysis is required to thoroughly describe the difficulty of the puzzle which likely has dependents on the implementation specifics of the Rosette framework and its solvers.

## Acknowledgments

I want to give warm thanks to the course instructor Prof. Yu Feng and, the teaching assistant Yanju Chen for the continuous support, guidance, dedication, and passion throughout the instructional process. I was able to gain an understanding of how solvers work, and how to use them to build cool programming tools. Prof. Yu Feng deserves my gratitude as I enjoyed every minute of his lectures and learned about automated reasoning techniques and their application in software.

## References

- [1] Port, Aaron C., and Roman V. Yampolskiy. "Using a GA and Wisdom of Artificial Crowds to solve solitaire battleship puzzles." In *2012 17th International Conference on Computer Games (CGAMES)*, pp. 25-29. IEEE, 2012.
- [2] Smith, Barbara M. "Constraint programming models for solitaire battleships." *University College Cork, Ireland, Available: <http://www.cs.standrews.ac.uk/~cppod/publications/reports/cppod-20-2006.pdf>* (2006).
- [3] Moshe Rubin, developer of Fathom It!, "What is Fathom It!" <http://www.mountainvistasoft.com/whatisfi.htm>
- [4] The Rosette Guide: [Rosette Essentials](#), [Angelic Execution](#)
- [5] Chen, S. T., S. S. Lin. 2004. Optimal algorithms for  $2 \times n$  Mastermind games—Graph-partition approach. *Comput. J.* 47(5) 602–611
- [6] Gedeon, T. D. 1996. An iterative solution produced by transformation. *Comput. J.* 39(4) 353–356
- [7] Majumdar, A. A. K. 1994. A note on the iterative algorithm for the Reves puzzle. *Comput. J.* 37(5) 463–464.
- [8] Sapir, A. 2004. The Tower of Hanoi with forbidden moves. *Comput. J.* 47(1) 20–24.
- [9] Sniedovich, M. 2002. OR/MS games: 2. Tower of Hanoi. *INFORMS Trans. Ed.* 3(1) 45–62.
- [10] Concept is Puzzles, Battleships, "Battleships techniques"; [Available](#)
- [11] Battleship puzzle. 2009. Wikipedia (the free encyclopedia); [Available](#)
- [12] M. Daly, "Detailed Solution Key for 2000 World Puzzle Championship Qualifying Exam, Problem 6," 2000, [Available](#)
- [13] Sevenster, Merlijn. "Battleships as a decision problem." *ICGA Journal* 27, no. 3 (2004): 142-149.
- [14] Pixel connectivity, February 2022, Wikipedia (the free encyclopedia); [Available](#)
- [15] GitHub repository for our code, Battleship Solitaire Solution Synthesis: [Available](#)